A Dynamic-Scene Gaussian SLAM Map: Anatomy, Compression, and Offline Repair

Anonymous cvm submission

Paper ID ****

Abstract

Dynamic-scene RGB-D SLAM systems built on 3D Gaussian splatting keep moving content out of the map, and the artifact they store and reload is that map, not the trajectory. Gaussian-map compression takes trained static or 4D models as input; the map a dynamic-scene SLAM system writes at run time has not been characterised as an input distribution. We study the map of one such system, a dense RGB-D splatting SLAM stack with an anti-dynamic mask. Opening the stored map shows 30% of its bytes to be bookkeeping the rasteriser never reads. A plain codec, uniform per-map quantisation with entropy coding and no learned parameters, compresses the frozen map $6\times$ at $\sim$0.01 dB of rendered PSNR loss and $15\times$ at an aggressive point, and compresses the repair operator's layered output to $\sim 8\times$ near-lossless; a post-hoc learned compressor lands within 10% of that point, buying little beyond the quantisation choice. Erasing dynamic content costs 17.9–22.2 dB in the erased regions; an offline repair operator restores 5.5–9.0 dB of it with no static-region damage beyond ± 0.25 dB, at 11–30% of the producing run's wall-clock. A second dynamic GS-SLAM system also stores non-unit quaternions and a weaker heavy tail, but not the bookkeeping composition or the near-isotropic scale bulk; a held-out panel from the same two benchmarks (BONN, TUM) reproduces the codec's working points.

Keywords: ***dynamic scenes, 3D Gaussian splatting, SLAM, map compression, offline repair***

1. Introduction

RGB-D SLAM built on 3D Gaussian splatting reconstructs a scene as a set of Gaussian primitives that can be rendered in real time. In dynamic scenes the central difficulty is deciding which pixels to trust. Moving people and objects violate the static-world assumption that the map optimises against. The standard answer is an anti-dynamic policy that keeps moving content out of the map, guided by a semantic mask.

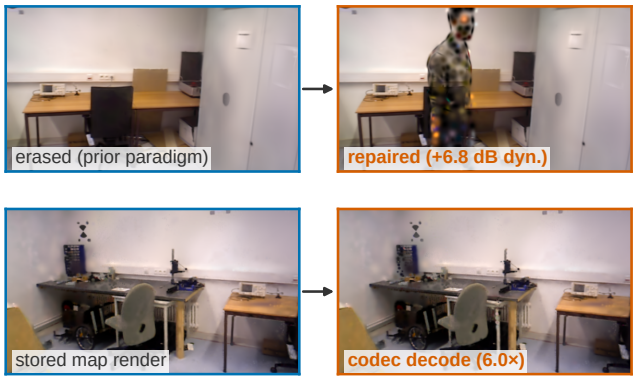


Figure 1. The anti-dynamic policy keeps online tracking robust, and it costs 17.9–22.2 dB in the erased regions (§6). Offline, on the frozen map, the codec compresses 6.0× at ∼0.01 dB loss, and the repair operator restores the erased dynamic content (+6.8 dB three-sequence mean: *crowd2*, *balloon2*, *person_tracking2*; static regions untouched; §7).

Dynamic-scene SLAM is judged primarily by tracking error. But the artifact that is stored, shipped and reloaded is not the trajectory; it is the map. This paper treats the map of a dynamic-scene splatting SLAM system as a first-class object (Fig. 1) and asks four questions about it. What is actually inside such a map? How small can a plain offline codec make it? What does the anti-dynamic policy cost on the map side? And can the dynamic content it erases be put back offline?

The system whose maps we study is the authors' prior work [2], accepted for publication at MMM 2027. It is a dense RGB-D splatting SLAM stack with an anti-dynamic mask, used as-is in every experiment. §3 describes it in full. To place the map study in a coordinate frame, we reproduce eleven published SLAM systems on the BONN dynamic benchmark under one protocol and report them alongside the stack in §3; the reproduction shows the regime this paper targets, in which most static methods diverge or degrade to tens of centimetres on dynamic sequences while dynamic methods, including the stack used here, stay accurate.

Within that frame we make three map-side contributions. First, we open these maps and find that 30% of their raw

bytes are constant bookkeeping the rasteriser never reads, that a quaternion redundancy survives in the fields it does read and that a near-isotropic scale bulk is available to a tighter coder, and that the heavy tail is load-bearing (§4). Second, we show that a deliberately plain codec, uniform per-map quantisation ranges with entropy coding and no learned parameters, reaches near-lossless $6\times$ compression at $\sim$0.01 dB loss and $15\times$ at an aggressive working point, with the gains traced to quantisation and verified on a held-out panel; the same scheme compresses the repair operator's layered output to $\sim 8\times$ near-lossless by exploiting temporal smoothness and rotation redundancy (§5). Third, we quantify the cost of the anti-dynamic policy. Erasing dynamic content costs 17.9–22.2 dB in the erased regions, most of it recoverable by inserting the dynamic geometry back into the map (§6); and we show that an offline repair operator that moves dynamic-content management out of the online loop restores the erased content with 5.5–9.0 dB of PSNR gain and no static-region damage beyond the ± 0.25 dB band (§7).

2. Related Work

Optimisation-based splatting SLAM. 3D Gaussian splatting [12] has become the dominant map representation for dense SLAM, replacing implicit fields with primitives that render in real time. SplaTAM [11] introduced Gaussian maps for RGB-D SLAM with a silhouette-based densification; MonoGS [18] formulated tracking and mapping as a joint photometric-geometric optimisation; Co-SLAM [25] is the corresponding implicit-field baseline on the same benchmarks. Compactness of the representation has been studied in this line. CGS-SLAM [7] and DiskChunGS [8] compress the Gaussian maps of static-scene SLAM systems, and importance-based pruning was introduced by PWAS [28] and by the significance-guided pruning of MGS-SLAM [5]. The codec here differs in the regime: the maps compressed come from a dynamic-scene system with an anti-dynamic policy. The heavy tail, however, is not confined to that regime (§4), so it is carried rather than clipped.

Dynamic-scene handling. Dynamic content breaks the static-world assumption of splatting SLAM. The dominant answer is to detect and erase. DynaSLAM [3] inpaints dynamic regions in a classical pipeline, NGD-SLAM [30] masks them on an ORB-SLAM3 base and runs on CPU only, RoDyn-SLAM [10] does the same with a neural implicit field, and among splatting systems DG-SLAM [26], WildGS-SLAM [31], GGD-SLAM [17], RGD-SLAM [24], Dy3DGS-SLAM [15] and BDGS-SLAM [27] mask or down-weight dynamic primitives during tracking and mapping. DGS-SLAM [13] is a separate, contemporaneous system whose name differs from DG-SLAM by one letter; only DG-SLAM is reproduced here. The opposite answer is to model the dynamics. DynaGSLAM [16] manages a separate set of dynamic Gaussians with a lifecycle, and the 4D line [9, 29] extends the representation over time. The repair operator of §7 adopts the DynaGSLAM management scheme but moves it offline, applied to the finished map as a post-process instead of inside the tracking loop.

Compact and compressible maps. Beyond SLAM-specific compression, 3DGS compression is an active area. HAC [6] and its successors learn entropy models over quantised attributes, Compact3D [14] and the compression survey [1] catalogue attribute-wise schemes, SOG [19] clips attribute ranges by percentiles, and StitchGS [23] prunes high-frequency SH parameters and quantises the rest. TC3DGS [9] compresses dynamic scenes by temporal grouping, and MEGA [29] by memory-efficient 4D representation. These methods operate on trained static or 4D models; the maps of a dynamic-scene SLAM system, with their constant bookkeeping fields and load-bearing heavy tail, are a different input distribution, which is the gap §5 addresses with a parameter-free codec.

Evaluation practice. Dynamic-scene SLAM is evaluated on the TUM [22] and Bonn [21] benchmarks, and the dominant metric is ATE. Rendering quality is reported as full-frame PSNR/SSIM/LPIPS following the protocol of MonoGS [18]. We additionally report metrics partitioned over the dynamic and static regions throughout; the protocol is stated once in §3. In contrast to the lines above, this work studies the map of a dynamic-scene system as the object of interest. We open it, compress it with a plain codec, quantify what the anti-dynamic policy erases, and repair the erased content offline.

3. System and Comparison

The system studied in this paper is a dense RGB-D splatting SLAM pipeline for dynamic scenes. Each frame is tracked against the current Gaussian map; the map then grows under photometric and depth supervision, while dynamic content is excluded from the map by a semantic mask (Fig. 2). The base design follows an optimisation-based RGB-D splatting SLAM [18], extended with four components: a Huber robust loss ($\delta = 0.1$) on the tracking residual, a per-pixel reliability signal, keyframe selection with a coverage gap cap of five frames, and a deferred-commit lifecycle that prunes map primitives. We report this stack as the *Prior RGB-D stack*. It is the authors' prior work [2], accepted for publication at MMM 2027, used as-is in every experiment of this paper; its measurements are placed here only as a coordinate frame, not as a result of this paper. The prior work ships two configurations, and every map-level number in this paper is measured on both: the *baseline*

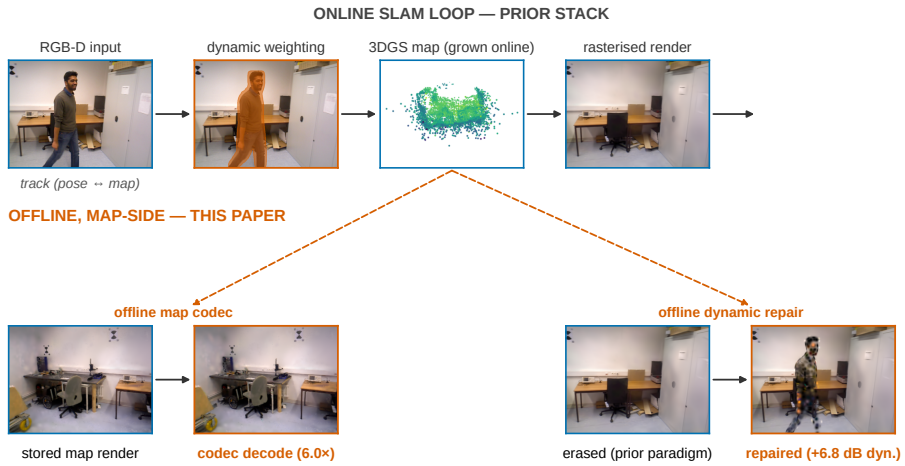


Figure 2. System overview with real artifacts (*crowd2* frame 621). Online (top, prior stack). Each frame is tracked against the current Gaussian map, dynamic content is masked rather than modelled, and the map grows under photometric and depth supervision. Offline (bottom, this paper). A codec compresses the stored map (6.0× over the 42 maps) and the repair operator restores erased dynamic content at the same poses (+6.8 dB, three-sequence mean: *crowd2*, *balloon2*, *person_tracking2*).

arm (the stack with the module switches off) and the *final arm* (the same stack with sliding-window pose correction and loop closure switched on). Both arms share the mask pathway, the keyframe schedule and the map schema, so a map from either arm is the same kind of object; we report them as two arms of one system throughout, never as two systems.

To make that coordinate frame concrete, we reproduce eleven published SLAM systems on the BONN dynamic benchmark [21]: the same machine, the same evaluation harness, and Horn SE(3) alignment with the error in centimetres (mean and standard deviation over three seeds; the protocol is stated once in the evaluation paragraph below and the raw outputs are released at publication). Execution conditions vary across systems: DynaSLAM and NGD-SLAM are classical keypoint pipelines (DynaSLAM on an ORB-SLAM2 base, NGD-SLAM on an ORB-SLAM3 base, both CPU-only), WildGS-SLAM is monocular, and seed reporting follows each system's capability. These numbers establish the tracking regime this map study builds on, not isolated map-architecture comparisons. Table 1 reports the absolute trajectory error (ATE) on the six BONN sequences shared between the reproduced systems and the prior stack (*balloon*, *balloon2*, *crowd*, *crowd2*, *person_tracking*, *person_tracking2*); the reproduction covers eight sequences per system, and the two extra ones available only to the baselines (*mv_no_box*, *mv_no_box2*) as well as the *crowd3* sequence available only to the prior stack are kept out of the table to avoid mixing incomparable subsets.

The table makes the regime of this paper visible in one glance. Most static SLAM methods reproduced under the same protocol fail outright on the dynamic sequences (Co-SLAM [25] exceeds 10^4 cm of ATE on *crowd* and *crowd2*, marked as diverged), and the rest of the static group degrades to tens or hundreds of centimetres in most cells, whereas the dynamic methods mostly stay in the single digits. The separation is not absolute, and we do not claim it as one. ORB-SLAM3, a static method, holds 7.27 ± 1.45 cm on *balloon*; within the dynamic group, RGD-SLAM reaches 20.10 ± 2.55 cm on *person_tracking2* and DynaGSLAM 23.82 cm on *crowd*, so both groups contain failures. The prior stack sits inside the dynamic group on its ATE (3.25 ± 0.07 cm on *balloon* up to 11.31 ± 0.70 cm on *person_tracking*; full per-sequence values in the table), which is the regime this paper targets. The maps studied in §4 and compressed in §5 come from a system that keeps dynamic content out of the map rather than modelling it. The higher readings on *person_tracking* and *person_tracking2* carry a global scale drift of ∼10–15%, which Sim(3) alignment removes with the same trajectory shape (bringing them to ∼7.6 cm on both sequences). We report the SE(3) numbers to stay on the same alignment as the reproduced systems, and flag the scale drift only so that the reading is not mistaken for a shape error (Fig. 3 shows a representative trajectory against ground truth).

Rendering quality and runtime, which the table does not report, follow the same split less cleanly. Among the eight reproduced systems with a reported rendering path, the three 3DGS dynamic methods reach a full-frame PSNR spanning 6.97–25.14 dB (DynaGSLAM 6.97–22.14 dB, DG-SLAM 19.75–23.67 dB, RGD-SLAM 22.76–25.14 dB), while the other dynamic methods and the static methods fall in 14–20 dB with substantial overlap, for example WildGS-SLAM at 15.6–18.7 dB and Co-SLAM at 14.2 ± 0.7 dB on *crowd*; the prior stack renders at 21.8 dB on *balloon*. Three of the reproduced systems (DynaSLAM [3], NGD-SLAM [30],

Method	*Family*	balloon	balloon2	crowd	crowd2	person_tracking	person_tracking2
Static methods							
MonoGS	3DGS	39.32±1.01	22.05±1.55	86.47±18.34[d]	147.46±39.03[d]	44.83±9.06	43.85±8.55
SplaTAM	3DGS	36.50±0.53	37.47±2.87	135.37±36.15[d]	493.22±470.66[d]	92.52±18.93[d]	132.72±9.37[d]
Co-SLAM	implicit	27.10±0.97	35.10±4.07	35866.32±38265.29[d]	24905.40±42817.31[d]	75.50±6.52	71.26±2.90
ORB-SLAM3	classical	7.27±1.45	19.81±2.18	76.60±27.86[d]	111.41±14.75[d]	69.54±2.19	76.82±2.14
Dynamic methods							
DG-SLAM	3DGS	3.66±0.02	3.83±0.08	5.76±0.15	6.39±0.50	4.25±0.06	6.12±0.13
RGD-SLAM	3DGS	2.45±0.22	4.26±0.89	2.61±0.28	2.36±0.06	7.21±1.24	20.10±2.55
WildGS-SLAM	3DGS	2.75±0.07	2.42±0.04	1.55±0.06	2.17±0.05	3.63±0.08	3.09±0.03
DynaGSLAM	3DGS	8.36	9.36	23.82	13.73	8.94	13.69
DynaSLAM	classical	2.88±0.16	2.59±0.10	2.07±0.21	2.85±0.28[d]	4.53±1.81	8.82±1.18
NGD-SLAM	classical	2.82±0.07	2.46±0.08	1.85±0.16	2.22±0.07	4.97±0.44	6.63±0.43
RoDyn-SLAM	implicit	8.47±0.31	11.90±0.21	5.60±0.18	4.63±0.06	16.39±0.54	14.36±0.11
Prior RGB-D stack	**3DGS**	3.25±0.07	5.48±0.27	2.23±0.10	2.26±0.32	11.31±0.70	10.40±0.28

Table 1. Absolute trajectory error on the common BONN sequences (RMSE, cm, mean±sd over 3 seeds; all-frames, Horn SE(3) alignment). Family is the map representation. The first eleven rows are a single same-machine reproduction. DynaGSLAM delegates tracking to an external front-end (DynoSAM poses) and its three seeds are therefore deterministic, so it is reported without variance. The last row is the prior RGB-D stack, evaluated on the same machine and protocol in an independent batch; it is reported as background and is not a result of this paper. Execution conditions vary across systems: DynaSLAM and NGD-SLAM are classical keypoint pipelines (DynaSLAM on an ORB-SLAM2 base, NGD-SLAM on an ORB-SLAM3 base, both CPU-only), WildGS-SLAM is monocular, and seed reporting follows each system's capability; these numbers establish the tracking regime this map study builds on, not isolated map-architecture comparisons. Static methods degrade severely on the dynamic sequences, which is the regime this paper targets; they are shown to place the dynamic methods in context, not as straw men. [d]: tracking diverged (path length $> 5\times$ GT or ATE > 100 cm) on initial run; re-run attempted under the same protocol with recommended settings. Values shown are from the re-run (diverged status retained where re-run also diverged). PSNR and FPS are in the text.

Sequence	PSNR (dB)	SSIM	LPIPS
balloon	21.80	0.852	0.273
balloon2	18.72	0.808	0.316
crowd	19.92	0.829	0.274
crowd2	19.50	0.823	0.297
crowd3	17.04	0.778	0.345
person_tracking	22.68	0.860	0.264
person_tracking2	22.72	0.861	0.249

Table 2. Full-image rendering quality of the prior stack, offline rendering protocol sampled away from the keyframe grid. This is the full-image protocol; partitioned static-region numbers are in Table 7. LPIPS uses the standard AlexNet backbone; lower is better.

Sequence	FPS excl.	FPS conc.	excl./conc.
balloon	1.162	0.645	1.80×
crowd2	1.278	0.987	1.29×
person_tracking	1.128	0.879	1.28×
median	1.162	0.879	1.32×

Table 3. Online-phase efficiency of the prior stack (baseline arm). The two FPS columns are a single-variable comparison: the same code revision and the same three sequences, differing only in processes per GPU (one vs. two); the per-sequence ratio spans 1.28–1.80, so concurrency is real but modest, not the several-fold penalty a cross-revision comparison would suggest. FPS = frames / online time, one seed per sequence; the median row is over the three sequences.

ORB-SLAM3 [4]) have no full-frame rendering path under the reproduction protocol, so we report only the numbers that exist. End-to-end frame rates range from 0.2–2 FPS for the splatting and NeRF systems to 50–72 FPS for the feature-based NGD-SLAM and ORB-SLAM3; these are same-machine measurements, not a throughput benchmark. Table 2 gives the prior stack's own full-image PSNR, SSIM and LPIPS per sequence, and Table 3 its online efficiency; per-system values are in the supplementary material (Tables S3–S4).

Two limitations are intentional. First, the baselines and the prior stack were evaluated in separate batches; they share the machine and protocol, and the caption marks this. Second, we report the diverged rows as they are instead of clipping them. A static method that loses track on a dynamic sequence is exactly the failure mode the anti-dynamic policy addresses.

Evaluation protocol. All measurements in this paper follow one protocol, stated here once and referenced from each study. Tracking accuracy is reported as ATE RMSE in centimetres under Horn SE(3) alignment, all frames, mean and standard deviation over three seeds (raw outputs released at publication). Rendering metrics use the same offline renderer for every arm and every sequence: a fixed evaluation frame set, the canonical dynamic masks (single source

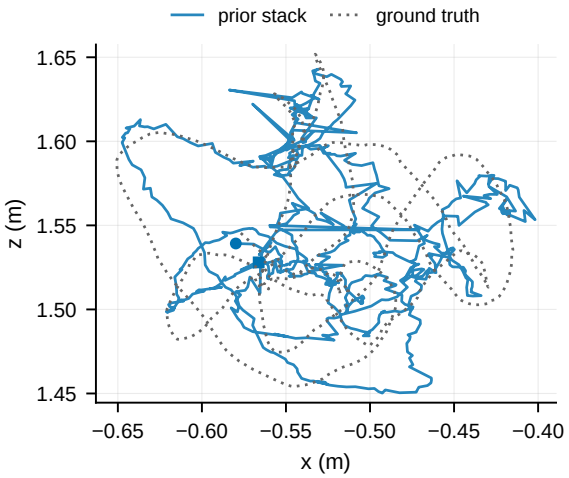


Figure 3. Trajectory of the prior stack on *crowd2* against ground truth (SE(3) alignment, world frame). The stack tracks the dynamic sequence without divergence, which is the regime the map study builds on.

of masks, rebuilt by one builder), and the full-frame plus region-partitioned protocols described where each number is used. One reproduced system, WildGS-SLAM, is monocular and receives RGB only; every other system receives RGB-D. Full-image PSNR differs across tables only through the frame set. The codec tables (§5) render a fixed evaluation set shared by all codec arms, while the system tables (Table 2, Table 7) render a grid-away frame set of the same maps, so absolute PSNR values are comparable within a table but not across the two protocols. Numbers from different batches are never mixed in one table, and every measurement in this paper is generated from logged artifacts by a script rather than copied by hand.

4. What a Dynamic-Scene Gaussian Map Is Made Of

Before designing anything that consumes a SLAM map, we open the map produced by the prior stack on the dynamic sequences and ask what is actually stored in it. The stack is the authors' prior work [2], used here as the primary study object; a cross-system check on the anatomy only closes this section. The representation is the standard 3D Gaussian one. Each primitive carries position, opacity, scale, rotation and a colour term, stored on disk as 20 float32 values (80 bytes per primitive at SH degree 0; 14 of the fields, 56 bytes, are read by the rasteriser) [12]. The remaining 6 fields (24 bytes, 30% of the raw bytes) are bookkeeping that the renderer never reads. They are a static probability, an observation count, an unmapped score, and a surface normal. The maps themselves are small. Across the seven Bonn sequences the baseline maps hold 3.2–7.1$\times 10^4$ primitives, or 2.6–5.7 MB on disk in the 20-field form. Three findings fall out of opening the map this way.

Dead weight, already paid for by the entropy coder. The six bookkeeping fields are constant across every map we inspected. The static probability is identically 0.7, the observation count and unmapped score are identically 0, and the normal is identically zero (std $\approx$ 0, one unique value). They carry no information in these files. Two consequences matter for what follows. First, this is why a naive *byte*-level view of a SLAM map overstates its content. 30% of it is schema, not scene. Second, the redundancy is already gone after the first compression pass. Entropy-coding the 20 float fields instead of the 14 renderer fields costs only 1.04$\times$ in rate. The dead fields are therefore an accounting fact about these map files, not a free compression gain.

Redundancy inside the fields the renderer does read. Two structural redundancies survive in the renderer-facing fields, and they differ in how far this paper takes them. The rotation is stored as a raw quaternion that the rasteriser renormalises at render time; across the maps we measured, 88.8–94.7% of the quaternions deviate from unit norm by more than 1%. The codec therefore stores three components and recovers the fourth from the unit constraint, which costs 0.29 dB at matched compression ratio in the layered codec of §5 (Table 5, row 2) and is used in the map codec as well. The scale is nearly isotropic for a large part of the map, but how large a part depends on the scene. For 78–92% of the primitives in the dynamic maps studied here the ratio between the largest and smallest scale axis is below 0.1 in log domain; on the two clean static sequences the share falls to 52–76%, and it is higher wherever people are present (81–85% walking, 89–91% seated). Storing the mean scale plus per-primitive deviations only for the anisotropic minority would save 23–48% of the scale group's bits depending on the working point (23–34% at 8–10 bits, 34–48% at 12 bits), measured on the dynamic maps. Near-isotropy is thus a property of these maps under dynamic content, not of 3D Gaussian maps in general: on DynaGSLAM's maps 0.0–0.1% of primitives qualify (median axis ratio 2.30–2.56 in log domain, against 0.003–0.008 in the dynamic maps here). The implementations differ in more than one respect (loss terms, densification, time parameterisation) and we did not ablate them; a fixed isotropic regulariser in the mapping loss is a candidate mechanism, not an established cause. The map codec studied here does *not* implement this second form: the quantiser of §5 already absorbs part of the redundancy, and the measured end-to-end gain is small (0.3–2.2% of the total bitstream). Fig. 4 summarises the byte budget.

The heavy tail is load-bearing. The scale distribution is heavy-tailed in a way that a generic compressor would not expect. A robust range (0.1% quantiles) is 1.36–2.15$\times$ narrower than the full range for the scale group and 1.25–1.53$\times$ for positions across the fourteen maps we inspected in de-

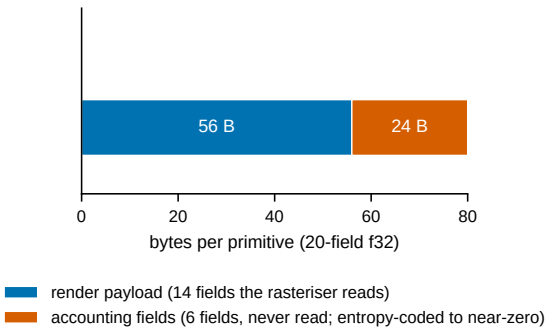


Figure 4. Byte budget of one map primitive (20-field float32, SH degree 0). The rasteriser reads 14 of the 20 fields.

tail (seven sequences × two arms, seed 0), and the tail is populated by genuinely large primitives. It is not a feature of dynamic content: it persists on the clean static maps (1.40–1.79×, twelve maps) and appears, weaker, in DynaGSLAM's maps (1.15–1.43×, twenty-four maps). The maximum-axis scale of tail primitives exceeds that of the bulk by at least two log units, with dozens of the largest primitives landing inside the tail (top-100 ∩ tail). The tail is not disposable. Clipping the dynamic range of the quantiser to the robust percentiles loses 0.71 dB of full-frame PSNR on *crowd2* (and 0.74 dB at the coarser working point), because the tail holds primitives that render; carrying the tail through with an escape code keeps it within noise of the full-range codec at a 5–15% rate premium. Range inflation is also not a reliable proxy for compressibility. The inflated groups are not the ones where robust ranges buy rendering quality. Fig. 5 shows the tail and the per-sequence inflation.

This map is therefore not what a generic compressor would assume. 30% of its raw bytes are constant schema, the rotation is safe to store in three components, the scale group has a near-isotropic bulk that a tighter coder could exploit, and the heavy tail must be carried. The second system keeps the non-unit rotation (96.5–98.6% of its primitives, against 88.8–94.7% here) and a weaker tail, but it stores only two quaternion components, so the byte-level saving does not transfer; its schema is its own (79 fields) and the near-isotropic bulk does not transfer. Section 5 builds a deliberately plain codec on these findings, and takes the rotation component of the first two.

5. A Simple Baseline Codec for Gaussian Maps

The anatomy of §4 suggests a compressor that any reader would write down first. Quantise each attribute group to a fixed bit width, entropy-code the result, and keep the heavy tail by choosing ranges that fit it. That is the codec we study, and the point of this section is that such a deliberately plain codec already reaches near-lossless 6× on the frozen dynamic-scene maps, with the gains traced to quantisation

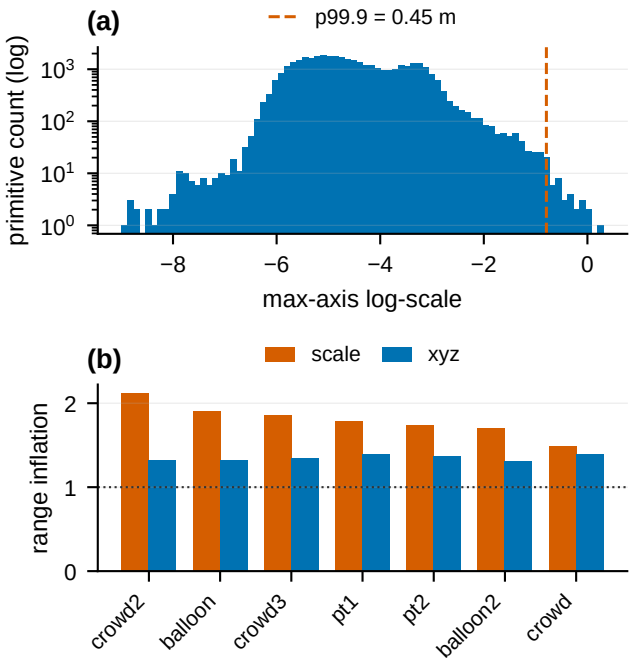


Figure 5. Heavy tail of a dynamic-scene map (top, max-axis log-scale histogram of *crowd2* with the p99.9 line) and per-sequence range inflation (bottom, robust vs full range for scale and position).

rather than to any learned component.

Design. Each primitive's attributes are grouped as position (xyz, three fields), colour (f_dc, three fields), opacity (one field), scale (three fields) and rotation. The rotation group is stored as the three non-leading quaternion components after normalisation, with the leading sign fixed non-negative and the fourth component recovered from the unit constraint, so the group carries three fields rather than four. Each group is quantised to a fixed bit width with a uniform range computed from that map's own data, and the quantised stream is entropy-coded with zlib. Writing a for one attribute of one group, the codec stores $q = \mathrm{round}\big((a-\ell)/(u-\ell)\cdot(2^b-1)\big)$ clipped to $[0, 2^b-1]$, where b is the group's bit width and ℓ, u are that map's own minimum and maximum for the group; decoding inverts the map. Both endpoints are carried in the header, not fitted to any panel. The bit widths are the only design choices and are the same three working points for every map, quoted in the order xyz/f_dc/opacity/scale/rotation: near-lossless 16/8/8/8/8, mid 10/6/6/6/6, aggressive 8/4/4/4/4. Only the per-map quantisation ranges are derived from each map's data, so nothing in the codec is fitted to the held-out panel. The codec is parameter-free. No learned weights anywhere. Decoding restores a standard PLY consumed by the same rasteriser as the source map.

Rate–distortion. We call a working point near-lossless when the full-image PSNR change stays within ± 0.25 dB of the source, the same band used for the static-region criterion in §7; the near-lossless point of Table 4 lands far inside it. Table 4 reports the rate–distortion behaviour on the frozen maps of the seven Bonn sequences (baseline and final arms, three seeds; 42 maps) plus a held-out panel of five TUM sequences and one Bonn sequence (36 maps) that never entered any design choice. The near-lossless point compresses to $5.96\times$ at -0.01 dB of full-image PSNR (full-image over the fixed codec evaluation set, including masked dynamic regions, §3); mid reaches $8.77\times$ and aggressive $15.11\times$ at -0.64 dB. The lossless generic compressors bound the free ratio at 1.37–1.62× (Table 4). We also run a post-hoc learned compressor on the same frozen maps. SOG [19] compresses unsorted Gaussians without retraining. On these maps its unquantised mode reaches $1.81\times$ at 0.00 dB and its quantised mode $6.53\times$ at -0.02 dB (per-configuration detail in supplementary Table S6), within 10% of the parameter-free codec's near-lossless point, and far below the 19.9–39.5× it reports on trained static scenes. What a learned codec buys here is thus almost entirely its quantisation choice. Niedermayr's pipeline [20] accepts a frozen map only by re-fitting it. 5000 supervised iterations reach $6.15\times$ at $+0.77$ dB. Methods that learn their entropy model jointly with the scene (HAC [6], Compact3D [14]) cannot consume a frozen map without per-map retraining.

Fig. 6 shows the corresponding rate–distortion curves per sequence.

Voxel merging, and where it breaks. Merging primitives closer than 2 cm in a voxel grid removes 20% of the primitives, which is why it appears as an ablation row rather than an operating point. The loss concentrates on the most crowded sequence, where the heavy tail of §4 carries the most weight and the anti-dynamic policy leaves the most erased content. Crowded dynamic scenes are precisely where primitives cannot be dropped, whether by merging or by pruning.

Alternatives we tried, briefly. Several further allocation and encoding schemes (region- or visibility-aware bit allocation, importance weighting, water-filling, trajectory-based pruning, Morton delta coding, isotropic-scale collapse) were implemented at matched rate and none improved the rate–distortion frontier. The closest is voxel merging, above. The redundancies of §4 are already partly absorbed by the entropy coder, and the heavy tail rules out range-based shortcuts.

Generalisation. The held-out panel (five TUM sequences: *freiburg1_desk2*, *freiburg1_room*, *freiburg3_long_office_household*, *freiburg3_sitting_rpy*, and *freiburg3_walking_rpy*; plus one Bonn sequence, *moving_obstructing_box*, never touched by any design choice) reproduces the working points: $5.96\times$ at near-lossless, independently computed on the panel's 36 maps (per-map ratios span 5.72–6.12; that the mean matches the frozen-map ratio of the same name is a numerical coincidence, the panels being disjoint), with a full-image PSNR of 15.90 dB against the source's own 15.90 dB, and 14.13 dB at aggressive. The bit widths and the range procedure were fixed before the panel was rendered, so this is an out-of-sample check, not a re-fit. The panel includes one dynamic TUM sequence (*freiburg3_walking_rpy*, a person walking in view); with a dynamic-region mask generated by the same pipeline as the Bonn maps, the near-lossless point loses at most 0.008 dB of PSNR in the dynamic region across six runs (two arms, three seeds), matching the static sequences. Split by scene class the working points barely move: $5.92\times$/$8.73\times$/$14.84\times$ on the two clean static sequences against $5.94\times$/$8.60\times$/$14.46\times$ on the dynamic two.

Cost. Encoding is a single-threaded CPU pass, taking 0.7 μs per primitive at the near-lossless point and 7.4 μs at aggressive, with decoding at 0.13–0.15 μs per primitive across all points. The coarse working points cost *more* to encode, not less: the quantiser itself is indexing-bound and takes the same time at every bit width, while the deflate pass runs longer on the lower-entropy stream. A map of tens of thousands of primitives encodes in tens to hundreds of milliseconds. The per-map quantisation ranges are a few dozen bytes of side information carried with the bitstream. The codec is offline by construction, so this cost never enters the online loop.

Compressing the repaired layers. The same scheme extends to the repair operator's output of §7, which is a time-ordered sequence of per-frame dynamic layers. Two findings of §4 carry over directly. The dynamic layers also carry a large share of suppressed primitives, whose opacity the offline mask pruning of §7 sets so low that they are invisible, and the same rotation redundancy is present. Three further mechanisms help in the temporal dimension. Primitives that persist across keyframes move smoothly (median per-step displacement 0.61 cm), so a constant-velocity predictor leaves a near-zero residual that quantises almost losslessly. Rotation is stored in three components with the unit constraint, and per-attribute bit widths give more bits to positions and fewer to opacity and rotation. The resulting codec reaches near-lossless $\sim 8\times$ compression of the layer sequence ($8.1\times$ on *crowd2*; 7.9–9.3× across four sequences), with the dynamic-region gain of §7 largely preserved (J1 kept at 84–96% of its uncompressed value across the four sequences, per sequence in Fig. 8). Fig. 7 shows the

Method	quant.	comp.	B/prim	full PSNR	Δ	static PSNR	held-out Δ
Source and lossless generic compressors							
source (uncompressed, 20-field f32)	–	1.00×	80	15.38	0.00	21.95	0.00
zlib (lossless)	–	1.47×	54.3	15.38	0.00	21.95	0.00
zstd (lossless)	–	1.54×	51.8	15.38	0.00	21.95	0.00
lzma (lossless)	–	1.62×	49.3	15.38	0.00	21.95	0.00
blosc2 (shuffle+delta) (lossless)	–	1.37×	58.2	15.38	0.00	21.95	0.00
Lossy baselines: generic (rate only) and post-hoc learned							
blosc2 TRUNC_PREC=16 (lossy)	–	2.84×	28.1	–	–	–	–
blosc2 TRUNC_PREC=8 (lossy)	–	3.80×	21.1	–	–	–	–
SOG (JXL, quantised)	–	6.53×	12.2	15.36	−0.02	21.82	−0.04
Niedermayr c3dgs (QAT 5k)	–	6.15×	13.0	16.15	+0.77	20.94	−0.01
This paper: uniform quantisation + zlib codec (lossy)							
near-lossless	16/8/8/8/8	5.96×	13.4	15.37	−0.01	21.79	−0.00
mid	10/6/6/6/6	8.77×	9.1	15.32	−0.06	21.64	−0.21
aggressive	8/4/4/4/4	15.11×	5.3	14.74	−0.64	19.21	−1.77
Ablation: + voxel merge (not an operating point)							
near-lossless + voxel merge	16/8/8/8/8	7.81×	10.2	15.30	−0.08	–	–
mid + voxel merge	10/6/6/6/6	11.26×	7.1	15.25	−0.13	–	–
aggressive + voxel merge	8/4/4/4/4	19.00×	4.2	14.72	−0.66	–	–

Table 4. Rate–distortion on the frozen dynamic-scene maps (seven Bonn sequences, baseline and final arms, three seeds) and an unseen held-out panel (five TUM sequences, one Bonn). Compression ratio is the mean over per-map ratios in each group; compression is the source 20-field f32 map size over the stored bitstream; B/prim is bytes per primitive; PSNR is rendered full-image (static = static region); Δ is the change from the source (negative = loss), held-out Δ on the held-out panel. Lossless compressors render identically to the source, bounding only the free ratio (≈ 1.4–1.6×); the gain above them is from quantisation, not entropy coding. The codec is lossy and parameter-free. The blosc2 rows are rate-only (rendered quality not measured). SOG shares the denominator and instrument, its bookkeeping fields carried losslessly (§5). The Niedermayr row is refit, not post-hoc; its reference implementation seeds nothing, so per-map values are not reproducible beyond ≈0.1 dB (§5). Voxel-merge rows are an ablation, not operating points (+26–31% ratio at 0.02–0.07 dB mean loss).

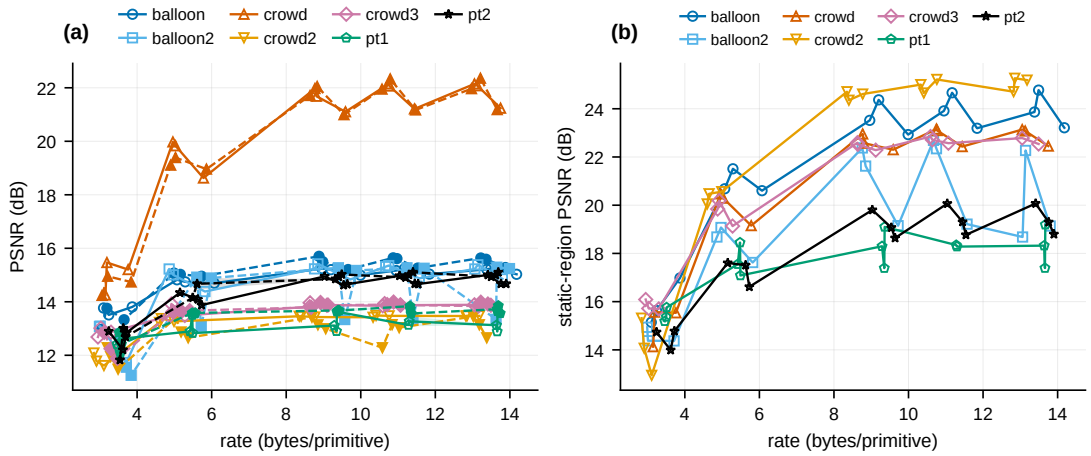


Figure 6. Rate–distortion curves on the seven Bonn sequences (three-seed mean, shaded band = interquartile range in panel (a) only). Left, full-image PSNR; right, static-region PSNR. Solid = baseline arm, dashed = final arm.

rate–distortion curve. A higher-fidelity 12-bit working point compresses the same layers to 3.9–4.7× while keeping J1 at 85–97% of its uncompressed value, passing the same criteria on all four sequences.

What each mechanism buys. On *crowd2*, where the ablation is measured, the four mechanisms play different roles. Suppressed-primitive exclusion is the largest lever. Without it the ratio drops from 8.12× to 6.20× on *crowd2*, because the invisible primitives would otherwise dominate the quantisation range. The three-component rotation saves a smaller but consistent 0.5× (7.66× without it), at a precision cost of 0.29 dB (J1 7.63 vs 7.92 dB, Table 5). The constant-velocity predictor does not save bytes at all (removing it raises the ratio to 8.33×); its role is precision, keeping J1 at 7.63 instead of 7.57 dB at the same bit budget. The per-attribute bit widths allocate the budget where it matters: positions keep the full width, opacity and rotation are quantised harder. Table 5 summarises the ablation.

Two boundaries are worth stating. The codec reads the

Configuration	*Ratio*	*J1 (dB)*
Full 10/6/4/6/6	**8.12**×	7.63
w/o prediction	8.33×	7.57
w/o rot3	7.66×	7.92
w/o suppression	6.20×	7.57
Uncompressed	1.0×	7.95

Table 5. Ablation of the 4D layer codec on *crowd2*. Removing the suppressed-primitive exclusion costs the most ratio; removing the rotation three-component form costs a smaller but consistent amount; removing the constant-velocity predictor saves no bytes (the ratio rises) and only costs a little precision. All variants keep the static region within criterion (J3 $\in \pm 0.25$ dB). J1 = dynamic-region PSNR gain vs the erased baseline.

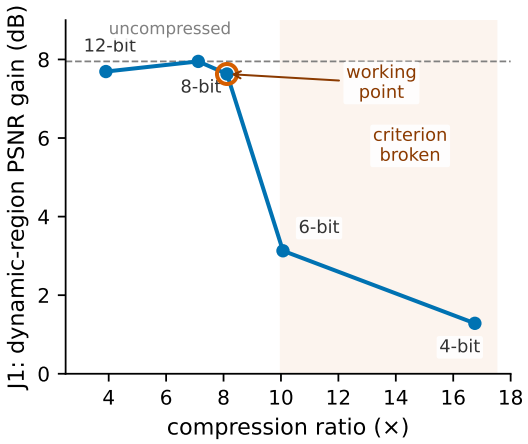


Figure 7. Rate–distortion of the 4D layer codec on *crowd2*. Dynamic-region PSNR gain (J1) against compression ratio at increasing bit widths. Low bit widths break the static-region criterion (shaded); the per-attribute working point 10/6/4/6/6 sits near the uncompressed gain at 8.1×.

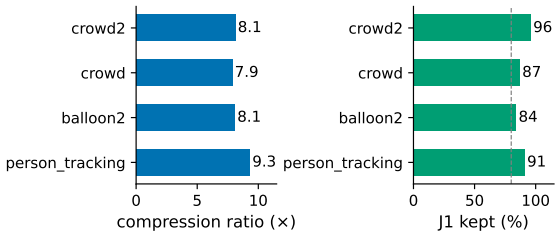


Figure 8. The 4D layer codec across four sequences at the 10/6/4/6/6 working point. Compression ratio (left) and the dynamic-region gain J1 kept after compression (right, percentage of the uncompressed value). The ratio is stable across sequences and the gain stays above 80%.

single merged PLY that the system saves, and the same quantisation applies to the layered output of the repair operator as described above. The absolute map sizes here are a few megabytes, so the honest unit of comparison is the ratio and bytes per primitive, not a headline megabyte number.

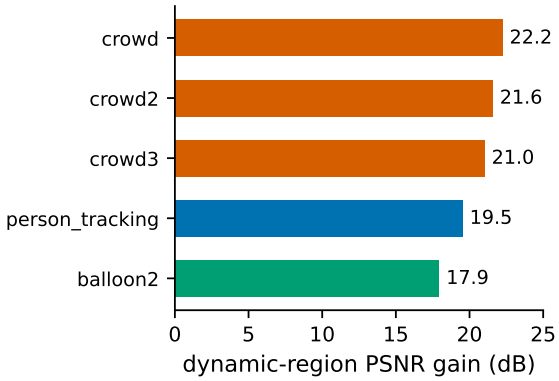


Figure 9. The erase cost is highest where the scene is most crowded. Dynamic-region PSNR gain when the erased content is re-inserted, per sequence (baseline arm). The three crowd sequences lose the most (21–22 dB), the single-object scene the least.

6. The Cost of the Anti-Dynamic Policy

The comparison of §3 establishes why dynamic scenes need an anti-dynamic policy at all. Static methods lose track outright. The policy itself, however, is not free. This section quantifies what erasing dynamic content costs on the map side, using the same frozen maps and the same offline renderer as §5, with the dynamic regions defined by the canonical masks and the erased content measured against a preserving counterfactual on the same frames.

Erasing dynamic content is expensive exactly where it matters. On five of the seven Bonn sequences (ten maps, excluding *balloon* and *person_tracking2*), re-inserting the dynamic content that the anti-dynamic policy erased (under ground-truth depth, as the ground-truth-depth oracle) improves the PSNR inside the dynamic regions by +17.9 to +22.2 dB and cuts the depth error there to 15–27% of its erased value, in every one of the ten maps (16–27% on the baseline arm and 15–24% on the final arm, Table 6; per-map values in supplementary Table S7). The gain is not an artifact of easy regions. It concentrates in the mask interior, where the erased map renders a static background over pixels the dynamic object should occupy (Fig. 10 and Fig. 11, two sequence types). Full-frame PSNR improves by +5.4 to +12.0 dB on the same runs. The cost scales with crowding. The three crowd sequences show the largest gains (+21 to +22 dB), the single-object and single-person sequences the smallest (+17.9 to +19.5 dB), and Fig. 9 shows the per-sequence spread.

Table 6 gives the per-sequence cost in full.

Most of the cost is recovered by geometry, not by motion modelling. A counterfactual decomposition on the same frames asks how much of the erased content can be recovered by inserting the dynamic geometry alone, with-



Figure 10. Erase policy at its most visible. Top, ground truth. Bottom, the map with dynamic content erased (prior stack). The vacated region left by the passing person is a static-background hole; the inset shows where the dynamic object should be.

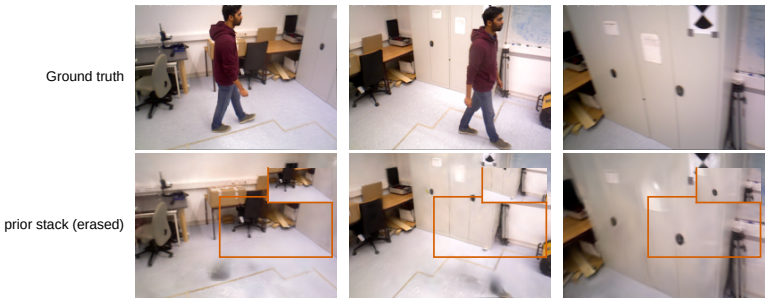


Figure 11. The same effect on a second sequence (*person_tracking*), three frames. Top, ground truth; bottom, the map with dynamic content erased. The erased map renders the static background over the region the moving person occupies, the same failure mode across sequence types; the orange inset marks where the person was.

Sequence	*Dyn. gain (dB)*	*Dyn. dL1 ratio*	*Full (dB)*	*Static (dB)*
crowd	22.2	0.19	+12.0	+0.29
crowd2	21.6	0.18	+10.6	+0.04
crowd3	21.0	0.19	+10.4	+0.39
person_tracking	19.5	0.27	+8.7	+0.28
balloon2	17.9	0.16	+5.4	+0.33

Table 6. The cost of the anti-dynamic policy per sequence (baseline arm). Columns: dynamic-region PSNR gain when the erased content is re-inserted; the depth-error ratio in the same region (lower is better); the full-frame gain; and the change in the static region. The crowd sequences lose the most in the dynamic regions and recover the most on re-insertion; no sequence damages the static region (all static changes are positive, +0.04 to +0.39 dB).

out modelling motion. The decomposition uses two probes, and we name them separately because their residuals are not the same quantity. The *ground-truth-depth oracle* inserts geometry unprojected from ground-truth depth at the density the map saw during mapping. It is an oracle ceiling: it measures what the erased content is worth, not what any system achieves. The *sensor-depth probe* inserts instead the geometry unprojected from the sensor depth the mapping stage actually saw, with no motion model, and is the realistic counterpart of that ceiling. Both insertions render with the map's own Gaussian parameters and lighting, so the ceiling bounds the recoverable content rather than reproducing the original frames. Within that ceiling, the sensor-depth probe recovers 0.80–0.93 of the oracle-recoverable depth error, leaving the dynamic-region depth error at 0.22–0.38 of its erased value, while freezing the inserted content at its first observation retains only 0.10–0.39. The dominant term of the anti-dynamic cost is therefore missing geometry, not missing motion. The erase policy pays for content it never inserts, and most of that content can in principle be put back by a static insertion. The insertion itself is cheap in map terms. It adds 6–29% additional primitives (median 22%). Section 7 then asks how much of this recoverable gain a realisable operator achieves under sensor depth and static preservation, and what mechanism pays for the difference.

The static side is not the price. The same decomposition confirms that the erase policy does not buy its dynamic-region savings by damaging the static map. Across the ten maps of both arms, nine change the static-region PSNR by +0.04 to +0.39 dB, a slight improvement, and the tenth by -0.03 dB, so no map degrades beyond the 0.25 dB band; the baseline arm alone is positive on all five sequences (Table 6). Table 7 reports the partitioned static metrics per sequence, including the ghost excess left where dynamic content was removed.

This is the motivation for §7. If the erased content is mostly geometry that can be inserted statically, an offline operator that moves the anti-dynamic management out of the online loop should be able to put it back. The accounting

Sequence	static PSNR	static SSIM	ghost excess (cm)	mask source
balloon	27.68	0.911	2.97	dynamic_mask
balloon2	26.28	0.902	−10.85	dynamic_mask
crowd	27.61	0.923	1.45	dynamic_mask
crowd2	27.45	0.922	4.43	dynamic_mask
crowd3	24.79	0.919	4.54	dynamic_mask
person_tracking	27.31	0.901	3.96	dynamic_mask
person_tracking2	27.74	0.893	2.21	dynamic_mask

Table 7. Partitioned metrics on the static support of the prior stack (M_{static} = valid GT depth ∧ ¬dynamic, frozen method-independent masks). All rows use the same canonical static protocol: offline 6-frame rendering with the `dynamic_mask` canonical masks (single-source masks, 84 runs). Ghost excess is the depth-L1 contrast between newly vacated regions and the static background; lower is better. A negative value means the newly vacated region renders slightly closer to the static background than the surrounding support does.

also fixes what to expect from such an operator. The erase cost of +17.9 to +22.2 dB is dominated by missing geometry, and the operator of §7 is measured against the ground-truth-depth ceiling, with the gap between the two attributed explicitly.

7. Recovering Dynamic Content Offline

§6 showed that most of what the anti-dynamic policy erases is geometry that a static insertion can recover. This section asks whether that recovery can be done offline, as a post-process on the finished map, by a repair operator that manages dynamic Gaussians the way an online dynamic-GS system would [16] but without running inside the tracking loop. The intended use is offline map refinement for applications that replay or ship a finished map, such as VR replay or archival reconstruction; the operator is not proposed as an online SLAM module. The static layer is frozen bit-for-bit; the operator only adds a separate dynamic layer, and the output is a time-ordered layer sequence rather than a single merged map.

The same mechanism, offline. Table 8 compares placements of the same dynamic-Gaussian management against the erase baseline. Offline, the operator recovers 5.49–8.95 dB of PSNR in the dynamic regions (J1, the dynamic-region PSNR gain over the erased baseline), cuts the depth error there to 0.23–0.61 of the erased value (J2), and leaves the static region at −0.04 to +0.20 dB (J3); 21/21 cells pass across seven sequences and three seeds, against pre-registered criteria of J1 ≥ 2.5 dB, J2 ≤ 0.70 and J3 within ±0.25 dB. (Fig. 12 shows the restoration on two sequences; per-cell values in supplementary Table S8; extra sequence in Fig. S1). These readings are deliberately below the ground-truth-depth ceiling of §6. The operator works from sensor depth and recomputed method masks instead of ground-truth depth, and it pays a static-preservation cost that trades some dynamic-region fidelity for a clean static region. Primitives projecting outside the method mask are suppressed, and because the layer fit has no scale upper bound, the fitted scales are clamped back to the insertion-time bound. The overlap between the operator's J2 range and the sensor-depth probe's residual range (0.22–0.38 of the erased depth error, §6) is expected rather than a contradiction. Both ratios share the same depth-error denominator, and the operator already recovers most of the recoverable error that the oracle quantifies. To separate the management stages from the insertion itself, we also run a control that keeps everything identical (same masks, same depth, same keyframe schedule, same fitting budget) but drops the lifecycle and static-protection stages, inserting each keyframe's observed geometry as new primitives. The control is not a useful operating point. It gains dynamic-region PSNR over the full operator (+0.8 to +2.0 dB) but destroys the static region (−11.7 to −16.7 dB, vs. ±0.25 for the full operator) and lets the dynamic layer grow without bound (∼10^5 to ∼10^6 primitives, ∼100× the operator's peak), because nothing prunes primitives whose object has left the view. The lifecycle and static-protection stages are therefore what make offline repair usable rather than merely high-gain. They buy a bounded layer and a clean static region, at the price of the dynamic-region margin reported above. The negative control below confirms that placement, not mere insertion, is required. The dynamic layer is a small addition to the map. Its peak size is 6.2×10^3 primitives on *balloon* against 3.2×10^4 in the frozen static map, so the repair adds roughly a fifth of the map's primitives rather than a second copy of it. The repair costs 0.11–0.30× the base run's online wall-clock (tracking plus mapping); the two quantities are different durations, so we report the ratio as a scale reference rather than as a like-for-like speedup. Where that time goes is dominated by a single stage. On *crowd2* the Adam fit of the dynamic layer takes 95% of the repair wall-clock (428 s of 449 s), while the mask unprojection is under 1%. The remaining time is snapshot and PLY writing. The offline operator is therefore cheap except for the fit, and the fit is what any placement of this mechanism must pay.

The gains are not an artifact of inserting content anywhere. A negative control that inserts the recovered dynamic geometry at a deliberately shifted camera position (C− arm, 0.3 m lateral shift) leaves the depth error in the dy-

Placement	J1 dynamic (dB)	J2 depth ratio	J3 static (dB)	Cost	Outcome
Online	$+6.97/+6.96$	0.35/0.26	$-0.49/-1.02$	$+95.1/+88.6\%$	fail
Erase (reference)	0	1	0	—	—
Offline	$+5.49$–$+8.95$	0.23–0.61	-0.035 to $+0.196$	0.11–0.30×	pass 21/21

Table 8. The same dynamic Gaussian management, run online and offline, against the erase baseline. J1/J2 are dynamic-region metrics, J3 static-region; online cost is per-frame mapping overhead, offline cost is the repair's share of the base run's online wall-clock. Offline is the full seven-sequence, three-seed grid (21 cells, all passing); online covers two sequences (crowd2, balloon2, seed 0). The comparison is descriptive, not single-variable: moving offline changes four things at once (pose source, static-layer freezing, keyframe schedule, mask pathway). The online dynamic readings use a snapshot instrument whose poses differ from the evaluated frame and carry a pose confound, so they are not comparable to the offline numbers.

namic regions 1.33–2.31× worse than the correctly placed repair across the seven sequences, so the recovered content has to land where the object actually was. The shift is small in absolute terms, but depth unprojection amplifies even a modest pose error into misaligned geometry. The negative control therefore confirms that the operator's gains come from correct placement under the recomputed masks, not from inserting plausible geometry in the general vicinity.

Offline versus online placement. The same mechanism online is not affordable. The online dynamic management measured on two sequences (*crowd2*, *balloon2*, seed 0) adds +95%/+89% per-frame overhead and damages the static region ($-0.49/-1.02$ dB), so the online form failed its own criterion and the offline placement exists at all. The overhead comes from per-frame mask recomputation, dynamic-layer fitting and lifecycle management running inside the tracking loop, where each frame must finish before the next arrives. The static damage is consistent with the online form not freezing the static map bit-for-bit; primitives near the dynamic boundary are updated during tracking, and lifecycle decisions made under tracking noise differ from those made over the finished trajectory. Moving offline breaks these constraints. The operator works from frozen poses and a frozen static map, fits the dynamic layer in batch with no per-frame deadline, and recomputes masks once per keyframe rather than once per tracked frame. The pose source, static-layer freezing, keyframe schedule and mask pathway all change together when moving offline, so the online and offline numbers are not single-variable comparisons. They are reported side by side only to show the direction of the trade-off, not as a causal claim. The offline placement pays a fraction of the base run's online wall-clock (0.11–0.30×) for a bounded layer and zero static damage; the online form pays roughly twice the per-frame cost for static degradation, and the offline placement is the one this paper evaluates.

Output form. The repair operator's output is a per-frame dynamic layer, not a rewrite of the base map. The single-merged-PLY form of the anti-dynamic output is exactly the form whose dynamic content is lost, so putting the recovered content back into it would erase the gain. The layer sequence keeps the static map bit-identical while the dynamic content lives in its own structure, which is why the static region reads zero damage. The cost of this form is that the output is not a drop-in replacement for the merged map; the codec of §5 compresses the merged PLY and the layer sequence alike, with the temporal mechanisms of that section tuned to the layered form (near-lossless ∼8×).

8. Discussion and Conclusion

What this paper is and is not. This paper studies a map, not a tracker. The system that produces the maps is prior work, reproduced and reported in §3 only to place the map study in a coordinate frame; its measurements are background, not a claim. Within that scope, the four map-side questions have concrete answers. The maps of the system studied here are 30% constant schema, two exploitable redundancies, and a load-bearing heavy tail (§4). A plain offline codec exploits these to reach near-lossless 6× compression and 15× at an aggressive working point, with the gains traced to quantisation and verified on a held-out panel (§5). The anti-dynamic policy that produces the map pays a measurable content cost, recoverable in large part by geometry alone (§6). And an offline repair operator puts the erased content back as a layer sequence, passing 21/21 cells across seven sequences and three seeds (§7).

Scope conditions. Four scope conditions delimit the claims of this study. First, the offline measurements use frozen maps and an offline renderer; they quantify what the map holds and what an offline operator can recover, not what an online system achieves; an online placement of the same mechanism shows higher cost under the snapshot instrument. Second, the repair output is a layer sequence, not a drop-in replacement for the merged map; the codec quantises both forms, but the layered form is consumed by its own temporal mechanisms and serves a different consumer than the merged PLY. Third, the evidence base is two indoor benchmarks (BONN dynamic, TUM); the held-out panel is drawn from the same two, so the readings are out-of-sequence but not out-of-benchmark. Within that

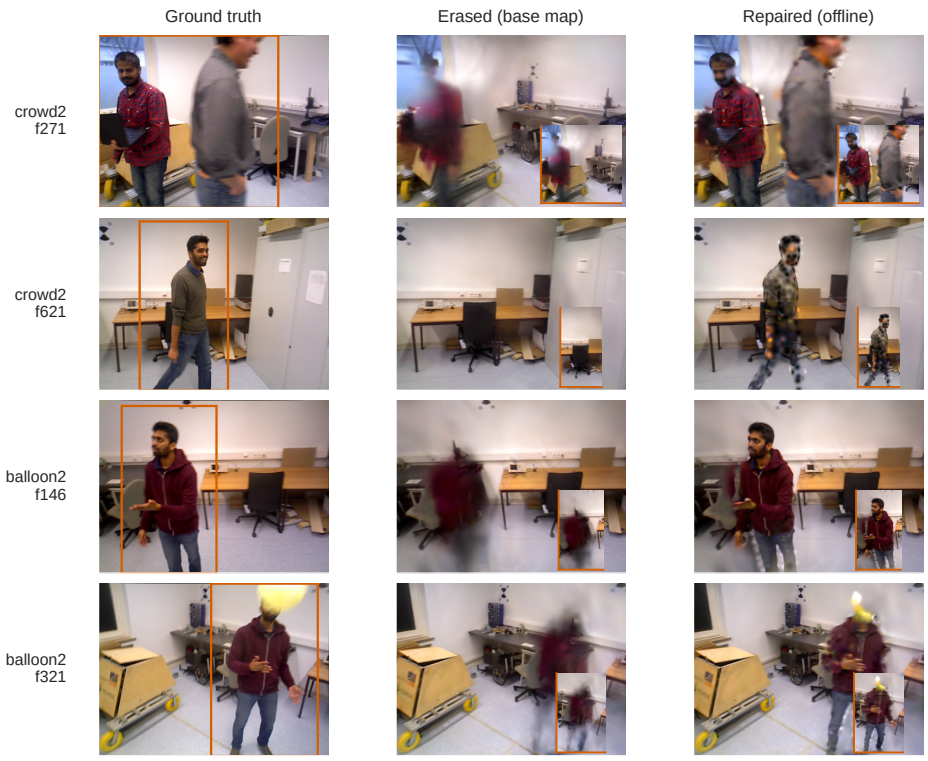


Figure 12. Offline repair on two sequences (crowd2, balloon2), two frames each, rendered at the evaluated frame's own pose. Columns are ground truth, the base map with dynamic content erased, and the base map with the dynamic layer restored. The erased people return without touching the static map; individual inserted primitives are visible at close zoom, a boundary of this repair path.

base, cost and repair come from one prior stack's maps; the anatomy adds a second system's, a cross-system check on the anatomy only. The codec's working points generalise to the held-out panel; the other readings establish what holds for these maps, not a cross-system law. Fourth, the absolute map sizes here are a few megabytes, so the honest unit of comparison is the ratio and bytes per primitive.

Limitations and future work. This study works within several constraints that future work can address. The evidence is RGB-D only; monocular splatting SLAM systems exist and one was reproduced in §3, but the repair operator requires depth to unproject geometry, and the ground-truth-depth oracle of §6 uses ground-truth depth, so the offline measurements are RGB-D throughout. The repair quality depends on the mask pathway inherited from the prior stack; better masks would tighten the dynamic-region bounds, and mask failures propagate directly into misplaced geometry. The layer sequence is not a drop-in replacement for applications that consume the merged PLY form; adoption would require a renderer or downstream tool that handles temporal layers, though the codec already compresses both forms. The anatomy, cost and repair readings come from one prior stack's maps; a second system's maps enter only the descriptive anatomy comparison of §4. Cross-system generalization of the cost and repair measurements would require reproducing them on other systems' maps, and whether the same recovery rates hold across architectures is an open question. Finally, an online system that manages dynamic content without erasing it remains unbuilt; the offline repair bounds what such a system could recover, but building an online form that maintains the static region while keeping dynamic content would require solving the per-frame overhead and coupling problems that make the online placement unaffordable here.

Why the map deserves attention. The map is what a dynamic-scene SLAM system stores, ships and reloads, and the object every downstream consumer touches. It does not look like what a generic compressor would assume, yet a deliberately plain codec compresses it well, and the anti-dynamic policy's cost is concentrated and recoverable. The repair operator points one step further: dynamic content can be managed outside the online loop, with the static map untouched.

Conclusion. We study the maps produced by a dynamic-scene RGB-D splatting SLAM system on their own terms. We open one, find the schema dead weight and the load-bearing tail, compress it to a sixth of its size at no visible loss

with a parameter-free codec, quantify what the anti-dynamic policy erases and how much geometry alone recovers, and restore the erased content offline across the full sequence-and-seed grid. The map of a dynamic-scene SLAM system is worth treating as a first-class object, and both its compression and its repair admit simple offline answers.

Availability. Code, map PLYs, tracking outputs, the codec, and the rendering and evaluation scripts are released at publication; every number is generated from logged artifacts by those scripts rather than copied by hand. During anonymous review the artifacts are available from the authors on request, and the anonymous version of the prior-work paper [2] is included with the supplementary material.

References

[1] M. S. Ali, C. Zhang, M. Cagnazzo, G. Valenzise, E. Tartaglione, and S.-H. Bae. Compression in 3D Gaussian splatting: a survey of methods, trends, and future directions. *IEEE Transactions on Circuits and Systems for Video Technology (TCSVT)*, 36(7):9291–9308, 2026. 2

[2] Anonymous Author(s). RoGS-SLAM: robust Gaussian splatting SLAM in dynamic environments via reliability-guided weighting, 2027. Accepted at MMM 2027; anonymous version included as supplementary material. 1, 2, 5, 14

[3] B. Bescos, J. M. Fácil, J. Civera, and J. Neira. DynaSLAM: tracking, mapping and inpainting in dynamic scenes. *IEEE Robotics and Automation Letters (RA-L)*, 3(4):4076–4083, 2018. 2, 3

[4] C. Campos, R. Elvira, J. J. Gómez Rodríguez, J. M. M. Montiel, and J. D. Tardós. ORB-SLAM3: an accurate open-source library for visual, visual-inertial and multi-map SLAM. *IEEE Transactions on Robotics*, 37(6):1874–1890, 2021. 4

[5] T. Cao, S.-S. Huang, K. Guo, H. Sha, D. Li, and Y. Liu. MGS-SLAM: monocular 3D Gaussian splatting SLAM with significance-guided pruning. *Computational Visual Media*, 12(4):941–957, 2026. 2

[6] Y. Chen, Q. Wu, W. Lin, M. Harandi, and J. Cai. HAC: hash-grid assisted context for 3D Gaussian splatting compression. In *European Conference on Computer Vision (ECCV)*, pages 422–438, 2024. 2, 7

[7] T. Deng, Y. Chen, J. Yang, S. Yuan, J. Liu, D. Wang, and W. Chen. CGS-SLAM: compact 3D Gaussian splatting for dense visual SLAM. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1606–1613, 2025. 2

[8] C. Feldmann, M. Wilder-Smith, V. Patil, M. Oechsle, M. Niemeyer, K. Tateno, and M. Hutter. DiskChunGS: large-scale 3D Gaussian SLAM through chunk-based memory management. *IEEE Robotics and Automation Letters*, 11(4):5009–5016, 2026. 2

[9] S. Javed, A. J. Khan, C. Dumery, C. Zhao, and M. Salzmann. Temporally compressed 3D Gaussian splatting for dynamic scenes. In *British Machine Vision Conference (BMVC)*, 2025. 2

[10] H. Jiang, Y. Xu, K. Li, J. Feng, and L. Zhang. RoDyn-SLAM: robust dynamic dense RGB-D SLAM with neural radiance fields. arXiv preprint arXiv:2407.01303, 2024. 2

[11] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten. SplaTAM: splat, track & map 3D Gaussians for dense RGB-D SLAM. In *CVPR*, pages 21357–21366, 2024. 2

[12] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4):1–14, 2023. 2, 5

[13] M. Kong, J. Lee, S. Lee, and E. Kim. DGS-SLAM: Gaussian splatting SLAM in dynamic environment. *arXiv preprint arXiv:2411.10722*, 2024. 2

[14] J. C. Lee, D. Rho, X. Sun, J. H. Ko, and E. Park. Compact 3D Gaussian representation for radiance field. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21719–21728, 2024. 2, 7

[15] M. Li, Y. Zhou, H. Zhou, X. Hu, F. Roemer, H. Wang, and A. Osman. Dy3DGS-SLAM: monocular 3D Gaussian splatting SLAM for dynamic environments. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 14572–14578, 2025. 2

[16] R. B. Li, M. Shaghaghi, K. Suzuki, X. Liu, V. Moparthi, B. Du, W. Curtis, M. Renschler, K. M. B. Lee, N. Atanasov, and T. Nguyen. DynaGSLAM: real-time Gaussian splatting SLAM for online rendering, tracking, motion predictions of moving objects in dynamic scenes. In *WACV*, pages 2434–2444, 2026. 2, 11

[17] Y. Liu, H. Xu, H. Duan, K. Fan, Z. Zhang, P. Zhuang, P. Luo, and H. Liu. GGD-SLAM: monocular 3DGS SLAM powered by generalizable motion model for dynamic environments. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 16900–16907, 2026. 2

[18] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison. Gaussian splatting SLAM. In *CVPR*, pages 18039–18048, 2024. 2

[19] W. Morgenstern, F. Barthel, A. Hilsmann, and P. Eisert. Compact 3D scene representation via self-organizing Gaussian grids. In *European Conference on Computer Vision (ECCV)*, pages 18–34, 2024. 2, 7

[20] S. Niedermayr, J. Stumpfegger, and R. Westermann. Compressed 3D Gaussian splatting for accelerated novel view synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10349–10358, 2024. 7

[21] E. Palazzolo, J. Behley, P. Lottes, P. Giguère, and C. Stachniss. ReFusion: 3D reconstruction in dynamic environments for RGB-D cameras exploiting residuals. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7855–7862, 2019. 2, 3

[22] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of RGB-D SLAM systems. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 573–580, 2012. 2

[23] J. Su, S. Pan, H. Zhu, S. Chen, Y. Huang, and Y. Zhou. StitchGS: towards seamless and lightweight large-scale 3D Gaussian splatting. *Remote Sensing*, 18(10):1460, 2026. 2

[24] H. Wang, Y. Shou, L. Shen, S. Li, and Y. Cao. RGD-SLAM: robust Gaussian splatting SLAM for dynamic environments. *Pattern Recognition*, 175:113071, 2026. 2

[25] H. Wang, J. Wang, and L. Agapito. Co-SLAM: joint coordinate and sparse parametric encodings for neural real-time SLAM. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13293–13302, 2023. 2, 3

[26] Y. Xu, H. Jiang, Z. Xiao, J. Feng, and L. Zhang. DG-SLAM: robust dynamic Gaussian splatting SLAM with hybrid pose optimization. arXiv preprint arXiv:2411.08373, 2024. 2

[27] T. Yang, S. Wei, J. Nan, M.-Y. Li, and M.-R. Li. BDGS-SLAM: a probabilistic 3D Gaussian splatting framework for robust SLAM in dynamic environments. *Sensors*, 25(21):6641, 2025. 2

[28] Z. Yang, H. Yin, L. Zhang, M. Liu, and A. C. Knoll. Importance-driven 3D Gaussian SLAM for efficient mapping and communication-aware sharing. In *CVPR Workshops (DriveX)*, pages 796–804, 2026. 2

[29] X. Zhang, Z. Liu, Y. Zhang, X. Ge, D. He, T. Xu, Y. Wang, Z. Lin, S. Yan, and J. Zhang. MEGA: memory-efficient 4D Gaussian splatting for dynamic scenes. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 27828–27838, 2025. 2

[30] Y. Zhang, M. Bujanca, and M. Luján. NGD-SLAM: towards real-time dynamic SLAM without GPU. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3467–3473, 2025. 2, 3

[31] J. Zheng, Z. Zhu, V. Bieri, M. Pollefeys, S. Peng, and I. Armeni. WildGS-SLAM: monocular Gaussian splatting SLAM in dynamic environments. In *CVPR*, pages 11461–11471, 2025. 2